



CIFRADO DE ARCHIVOS MULTIMEIDA

Compresión y Seguridad Práctica 1

Integrantes del grupo

Abel Martínez Molina
Reyes Llavador Castelló
Rodrigo Martínez Losada
Ionatan Florin Hudrea Colceriu
María Luisa Roca Vega

Tabla de contenido

Contenido del Archivo Comprimido _____	3
Como ejecutar la aplicación _____	3
Librerías y Software Utilizado _____	7
Comandos y Flujo de la Aplicación _____	11
Funcionamiento de la base de datos _____	14
Conclusión _____	15

Contenido del Archivo Comprimido

El archivo compartido constituye una estructura de carpetas que simulan el funcionamiento de nuestra aplicación. El proyecto parte de una carpeta raíz llamada Maquina que contiene los recursos necesarios para la ejecución de la aplicación, desde el código fuente hasta el archivo para importar la base de datos. Además, consta de dos carpetas principales: “Servidor” y “Local”, que reflejan la arquitectura de nuestra aplicación: el lado del servidor y el lado del cliente (local).

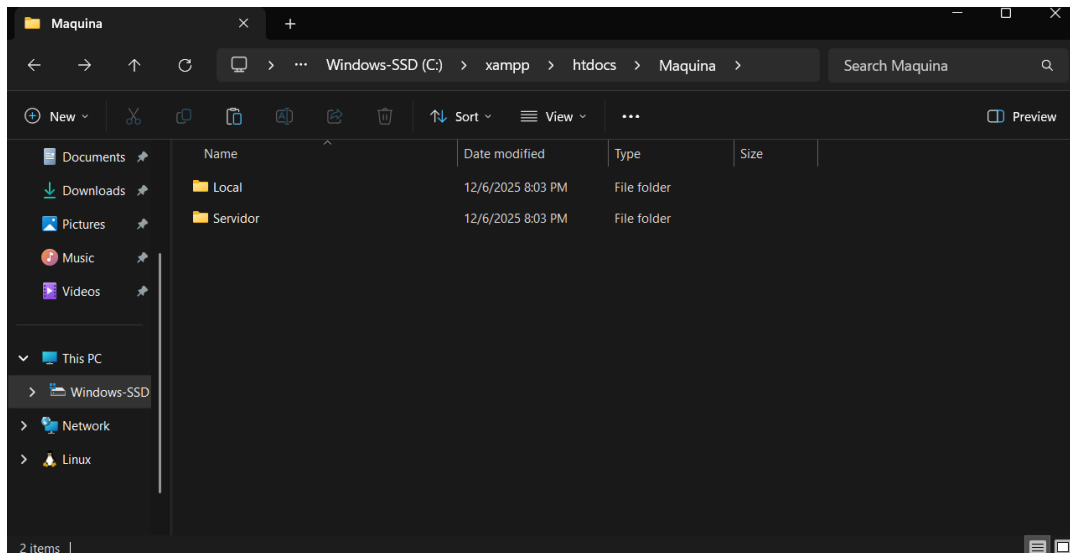
Carpetas Principales:

- **Servidor:** Esta carpeta contiene todos los archivos relacionados con el backend de la aplicación, es decir, el código que se ejecuta en el servidor. Los archivos principales que encontramos en esta carpeta son:
 - **DB:** Contiene el archivo con los datos iniciales para crear la base de datos cys en la herramienta XAMPP.
 - **PHP:** Contiene los scripts de PHP que gestiona la interacción con el usuario, la base de datos y las llamadas a la lógica de cifrado (prac1.class).
 - **Usuarios:** esta carpeta está compuesta por subcarpetas con los nombres de los usuarios que utilizan la aplicación y contiene los archivos originales encriptados que el usuario ha subido.
- **Local:** contiene los archivos necesarios para la interfaz de usuario (frontend) que se ejecuta en el navegador del usuario. Los archivos principales que encontramos en esta carpeta son:
 - **App:** Contiene la lógica y la presentación de la aplicación. Dentro incluye los archivos HTML (como dashboard.html, index.html, etc.), que definen la estructura de las vistas; la carpeta Style, que contiene los archivos CSS responsables del diseño y la apariencia visual; y la carpeta Js, donde se encuentra la lógica interactiva del lado del cliente (frontend), como login.js, register.js, entre otros.

Como ejecutar la aplicación

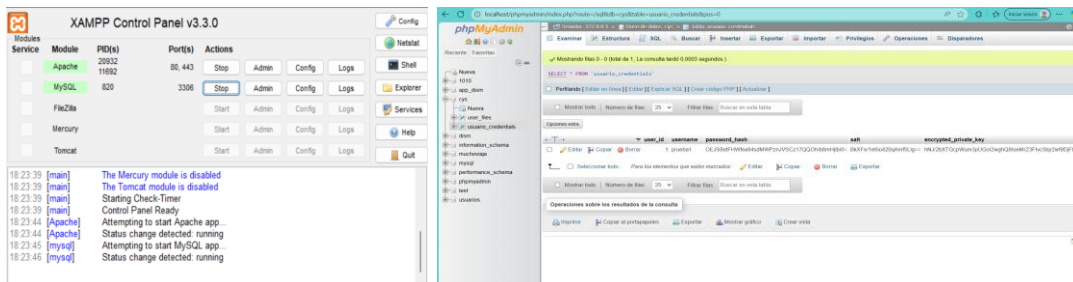
En este apartado vamos a comentar el proceso que se debe de seguir para poder utilizar nuestra aplicación.

1. **Preparar los archivos:** Descarga y descomprime el archivo ZIP. Obtendrás una carpeta llamada Maquina, que contiene todos los archivos necesarios para que la aplicación funcione. Luego, copia la carpeta test dentro de la carpeta “htdocs” de XAMPP.

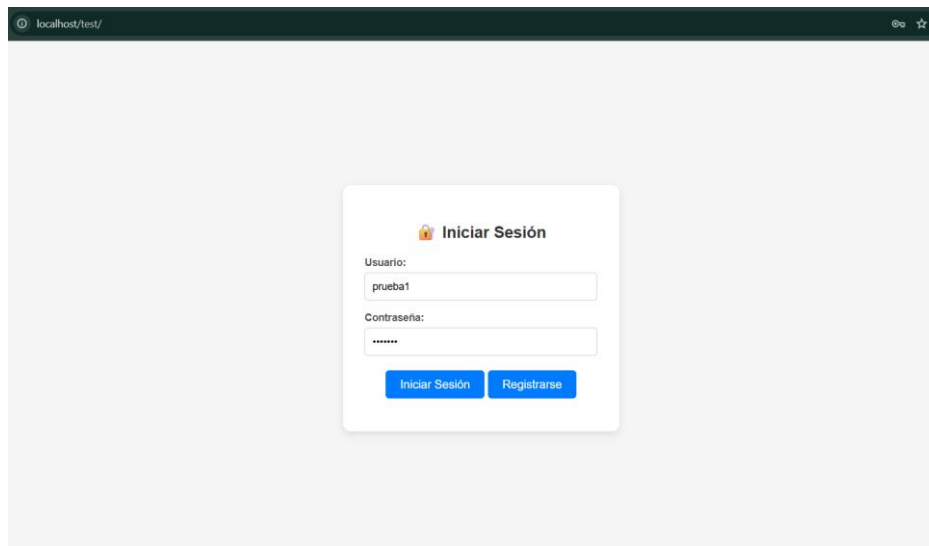


2. **Configurar XAMPP y la base de datos:** Para poder configurar el servidor deberemos seguir los siguientes pasos:

- Abre XAMPP y arranca los servicios Apache y MySQL.
- Accede a phpMyAdmin y crea una nueva base de datos llamada cys.
- Importa el archivo cys.sql, que se encuentra dentro de la carpeta db, para cargar la estructura y los datos iniciales.
- Además, tienes que dirigirte al documento php.ini en la carpeta XAMPP/php y descomentar 'extension=gd' (quitar el punto y coma (;) que tiene delante), luego apagar y encender XAMPP nuevamente.



3. **Acceder a la aplicación:** Para poder acceder a nuestra aplicación, deberemos acceder desde cualquier navegador a: <http://localhost/Maquina/Local/app>. Una vez accedido al puerto de xamp, lo primero que veremos será el inicio de sesión. En ella, podrás iniciar sesión o acceder a la página de registro. Si aún no tienes cuenta, crea una nueva y luego inicia sesión.



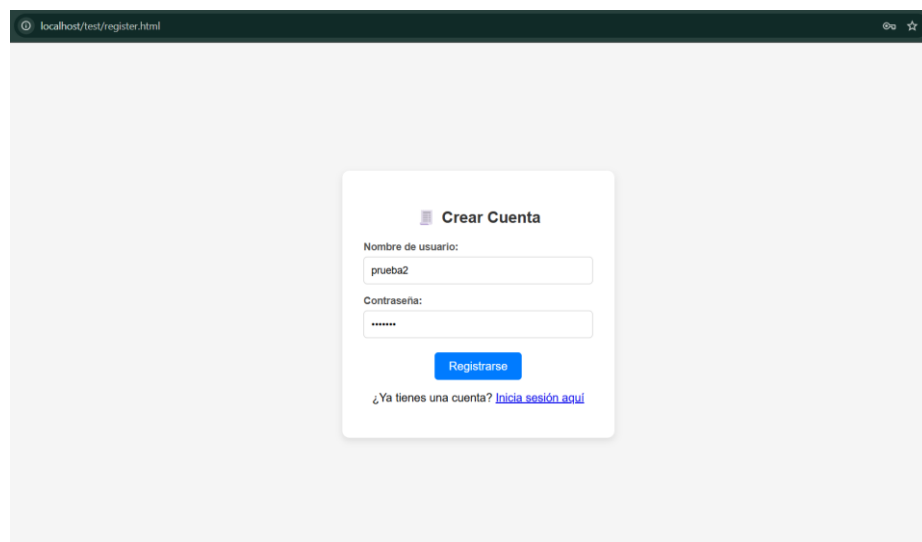
localhost/test/

Iniciar Sesión

Usuario:

Contraseña:

[Iniciar Sesión](#) [Registrarse](#)



localhost/test/register.html

Crear Cuenta

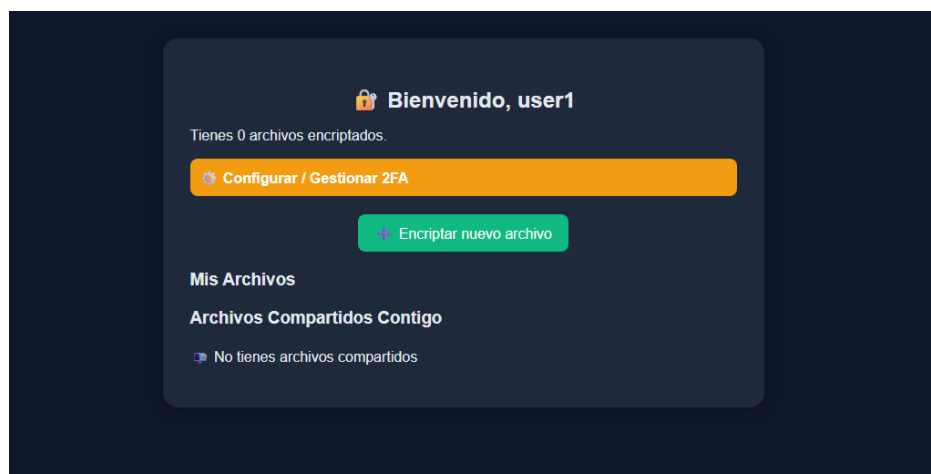
Nombre de usuario:

Contraseña:

[Registrarse](#)

¿Ya tienes una cuenta? [Inicia sesión aquí](#)

4. **Usar la aplicación:** En la página principal, podrás subir archivos (imagen, audio o video) para encriptarlos y compartirlos con otros usuarios. Además, si ya tienes un archivo encriptado, podrás desencriptarlo y descargarlo para acceder a su contenido nuevamente.



localhost/test

Bienvenido, user1

Tienes 0 archivos encriptados.

[Configurar / Gestionar 2FA](#)

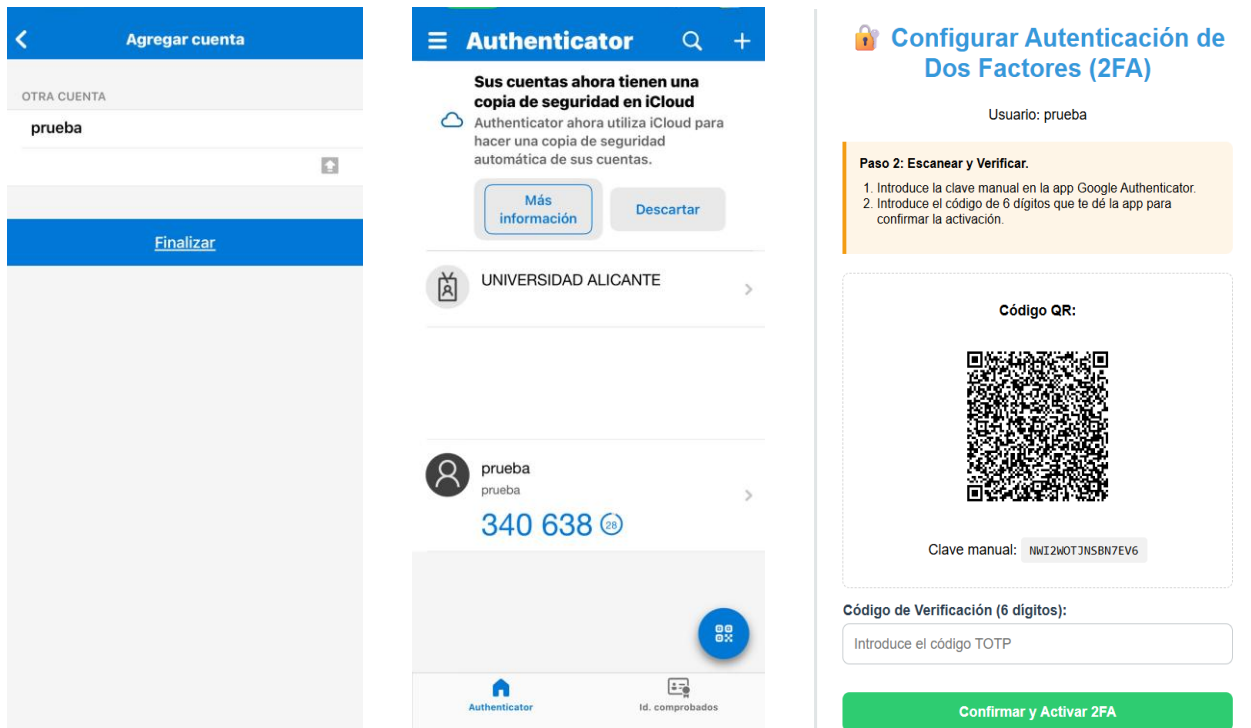
[Encriptar nuevo archivo](#)

Mis Archivos

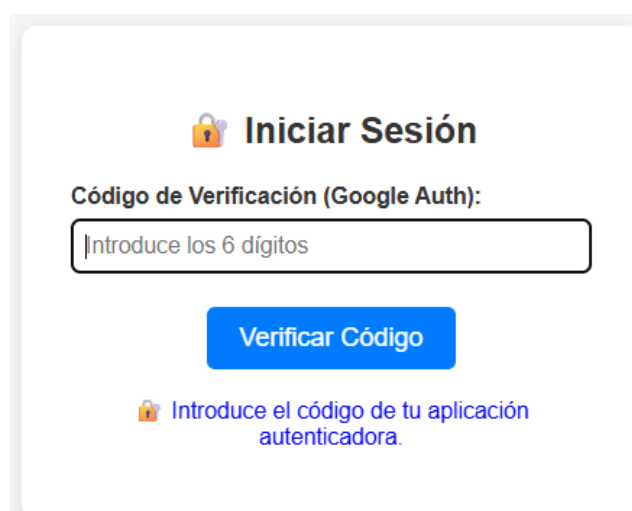
Archivos Compartidos Contigo

[No tienes archivos compartidos](#)

5. **Doble Factor de Autenticación:** Si deseas activar el doble factor de autenticación necesitaras generar una clave secreta pulsando en el botón. Esta clave se debetrá introducir a la aplicación 'Google Authenticator' junto a tu nombre de usuario.



Una vez hayas introducido los datos en la aplicación de Google Authenticator, esta misma te generará un código temporal que tendrá una validez de 30 segundos (TOTP). Ahora cada vez que un usuario inicie sesión, este deberá introducir el código temporal que la aplicación Google Authenticator proporciona al usuario para entrar a la aplicación.



Librerías y Software Utilizado

La aplicación utiliza una arquitectura de backend que combina PHP para la web y la gestión de la base de datos, Java para la criptografía de alto nivel, y HTML/JavaScript para el frontend.

- **Software de Servidor y Base de Datos:** XAMPP actúa como el entorno de desarrollo y ejecución de la aplicación, para ello utilizamos apache, que es el servidor web que aloja y sirve los archivos de la aplicación (HTML, PHP, JavaScript) a los navegadores de los usuarios. MySQL como sistema de gestión de la base de datos donde se almacenan las credenciales de los usuarios y los datos cifrados de los archivos. Y el lenguaje PHP de scripting que se ejecuta en el servidor y se encarga de la lógica de la aplicación (registro, inicio de sesión, manejo de subidas de archivos) y es crucial porque ejecuta los comandos de Java (prac1.java).
- **Lenguajes de Programación y ejecución:** Se usa el lenguaje Java como motor que ejecuta el código principal de seguridad (prac1.java). Este archivo es el responsable de todas las operaciones criptográficas que hacen la aplicación funcionar. Los lenguajes HTML, CSS, y JavaScript componen la interfaz de usuario (frontend). HTML (dashboard.html) proporciona la estructura, CSS le da estilo, y JavaScript (dashboard.js) gestiona la interacción del usuario en el navegador. PHP (además de ser un componente de servidor) contiene la lógica del backend para comunicarse con la base de datos y gestionar las sesiones de usuario.
- **Librerías Externas para el Doble Factor de Autenticación (2FA):** Para implementar el sistema de Doble Factor de Autenticación (2FA) compatible con aplicaciones como Google Authenticator, se utilizan dos librerías principales de PHP: una para manejar la lógica del secreto y el código temporal, y otra para generar el código QR que facilita la configuración al usuario.
 - Manejo de Google Authenticator (TOTP): Se utiliza la clase PHPGangsta_GoogleAuthenticator (archivo GoogleAuthenticator.php). Esta librería implementa el algoritmo TOTP (Time-based One-time Password, o Contraseña de un Solo Uso Basada en el Tiempo), que es el estándar utilizado por Google Authenticator.
 - Generación de Códigos QR: Se utiliza la clase Endroid\QrCode\QrCode (archivo QrCode.php), que es una librería moderna para la generación de imágenes de Códigos QR.

- **Librerías Criptográficas y Extensiones:**

- **Clases de Utilidad de Java:** Se utilizan clases estándar de Java como Base64 para codificar y decodificar datos binarios a texto, permitiendo su almacenamiento y transmisión segura en la base de datos y a través de la web.
- **Extensiones Criptográficas de PHP:** El código de inicio de sesión (login.php) utiliza funciones nativas de PHP como `hash_pbkdf2()` para verificar la contraseña del usuario de forma segura y `hash_equals()` para comparar hashes sin riesgo de ataques de tiempo.
- **Java Cryptography Extension (JCE):** Esta es la principal librería utilizada por el código Java (prac1.java). La JCE proporciona todos los algoritmos criptográficos necesarios, incluyendo:
 1. Cifrado AES-256 en modo GCM para la encriptación de datos rápida y segura.
 2. Cifrado RSA-2048 con padding OAEP para la protección de la clave de sesión AES.
 3. La función de derivación de clave PBKDF2 con HMAC-SHA256 para proteger la contraseña del usuario.

- **Creación de la firma digital:**

Para implementar la firma digital en el archivo a la hora de compartirlo con otro usuario se tienen que hacer unos pasos:

1. Obtener la información del archivo cifrado (los bytes del archivo cifrado). Se obtienen tal que así:

A screenshot of a code editor with a dark background and light-colored text. At the top left, there are three colored circles (red, yellow, green) representing window controls. The code shown is a single line: `1 byte[] encryptedFile = readFileToBytes(filePath);`. The text is in a monospaced font, with 'byte[]' in blue, 'encryptedFile' in green, and the rest in white.

```
1 byte[] encryptedFile = readFileToBytes(filePath);
```

Aquí se obtienen los datos cifrados a partir de un path proporcionado por el archivo PHP que llama al JAVA.

2. Ahora necesitamos la clave privada del dueño del archivo, eso se hace tal que así:

```
1 byte[] encPrivBytes = Base64.getDecoder().decode(user1EncPrivB64);
```

Aquí recibimos la clave privada en Base64 del archivo PHP y la pasamos a bytes (antes estaba en String ya que era Base64). Lo siguiente es obtener la clave privada en el formato PrivateKey de Java:

```
1 KeyFactory kf = KeyFactory.getInstance("RSA");
2 PKCS8EncodedKeySpec privSpec = new PKCS8EncodedKeySpec(privBytes);
3 PrivateKey priv = kf.generatePrivate(privSpec);
```

3. Ahora simplemente necesitamos un timestamp (marca de tiempo), que se obtiene de esta forma:

```
1 String timestamp = generateTimestamp();
```

Ahí se usa el método generateTimestamp(), que se muestra a continuación:

```
1 public static String generateTimestamp() {
2     DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")
3         .withZone(ZoneOffset.UTC);
4     return formatter.format(Instant.now());
5 }
```

Dónde se formatea el Instant.now() para no causar problemas relacionados con el formato a la hora de verificar la firma digital.

- Ahora que ya tenemos los 3 componentes necesarios (datos cifrados, clave privada del dueño y el timestamp) solo falta hacer la firma digital:



```
1 String signatureB64 = signFile(encryptedFile, priv, timestamp);
```

En el método `signFile()` lo primero que hacemos es iniciar el Signature de Java:



```
1 String signatureB64 = signFile(encryptedFile, priv, timestamp);
```

- Después escribimos los datos cifrados y el timestamp en un buffer (`ByteArrayOutputStream`):



```
1 ByteArrayOutputStream baos = new ByteArrayOutputStream();  
2 baos.write(fileData);  
3 baos.write(timestamp.getBytes("UTF-8"));  
4 byte[] dataToSign = baos.toByteArray();
```

- Para finalmente añadir el contenido del buffer en el Signature (que es dónde se hashea con SHA-256) y poder firmarlo con la clave privada:



```
1 signature.update(dataToSign);  
2 byte[] digitalSignature = signature.sign();
```

- **Comprobación de la firma digital:**

El método `verifySign` tiene como objetivo comprobar la autenticidad e integridad de un archivo mediante la verificación de su firma digital. Para ello, recibe como parámetros la clave pública del usuario que generó la firma (`senderPub`), el timestamp asociado al momento en que se firmó el contenido, la firma digital codificada en Base64 (`signatureB64`) y los datos del archivo en formato `byte[]` (`fileData`). El archivo ya ha sido descifrado previamente de manera local, y se envía junto con el timestamp para reconstruir la información original que fue firmada.

En primer lugar, se crea un objeto `Signature` utilizando el algoritmo `SHA256withRSA`. A continuación, el método inicializa dicho objeto en modo verificación mediante `sig.initVerify(senderPub)`, indicando que se utilizará la clave pública del emisor para validar la firma. Esto permite comprobar si la firma digital recibida fue generada con la clave privada correspondiente.

Posteriormente, se concatenan los datos del archivo y el timestamp dentro de un buffer `ByteArrayOutputStream`. Primero se escriben los bytes del fichero y luego los bytes del timestamp convertidos a UTF-8. Con esto se obtiene exactamente la misma estructura de datos que se utilizó originalmente para firmar: `archivo || timestamp`. Una vez reconstruidos los datos, se pasan al objeto `Signature` mediante `sig.update(...)`, permitiendo que internamente se calcule el hash sobre todo el conjunto.

Finalmente, se decodifica la firma digital recibida desde Base64 y se ejecuta `sig.verify(...)`. Esta llamada compara el hash calculado a partir de los datos proporcionados con el hash cifrado incluido dentro de la firma digital. Si ambos coinciden, significa que el archivo y el timestamp no han sido modificados desde su firma y que efectivamente fueron firmados con la clave privada asociada a la clave pública proporcionada. En ese caso, el método devuelve `true`. En otro caso, se devuelve `false`, indicando alteración o falsificación.

Comandos y Flujo de la Aplicación

La funcionalidad principal de la aplicación se ejecuta a través de comandos lanzados por PHP, donde la clase `prac1` se ejecuta con diferentes parámetros en función del modo de operación:

Modo registro:

- **Comando:** *`java prac1 register <password>`*
- **Funcionalidad:** Sirve para registrar un usuario y generar su material criptográfico inicial.
- **Funcionamiento:**
 1. Genera un salt (16 bytes aleatorios).

2. Genera un hash usando PBKDF2 y SHA-256 con 200 mil iteraciones y un output de 256 bits en base a la contraseña introducida.
3. Genera una clave AES derivada del hash.
4. Genera un par de claves RSA (2048 bits), claves pública y privada del usuario.
5. Salida JSON (en base64 para guardarlo en la base de datos):

```
{
  "salt": "...",
  "passwordHash": "...",
  "encryptedPrivateKey": "...",
  "publicKey": "..."
}
```

Modo Encrypt:

- **Comando:** `java prac1 encrypt <filePath> <password> <pubKey> <salt> <encPrivKey>`
- **Funcionalidad:** Sirve para cifrar un archivo.
- **Funcionamiento:**
 1. Obtiene los parámetros pasados por el comando decodificando de base64.
 2. Reconstruye el hash del password usando salt y pbkdf2 para poder reconstruir la clave AES.
 3. Reconstruye la clave AES.
 4. Desencripta la clave privada usando la clave AES.
 5. Reconstruye la clave privada con sus bytes.
 6. Reconstruye la clave publica con sus bytes.
 7. Genera una clave AES aleatoria.
 8. Convierte el fichero en bytes.
 9. Encripta el fichero con la clave AES generada aleatoriamente.
 10. Encripta la clave AES con la clave publica usando RSA-OAEP, SHA-256, y MGF1.
 11. Salida JSON (en base64 para guardarlo en la base de datos):

```
{
  "salt": "...",
  "passwordHash": "...",
  "aesKey": "...",
  "encryptedPrivateKey": "...",
  "encryptedFile": "...",
  "publicKey": "..."
}
```

Modo Decrypt:

- **Comando:** `'java prac1 decrypt <encryptedFilePath> <password> <aesKeyEncrypted> <pubKey> <salt> <encPrivKey>'`
- **Funcionalidad:** Sirve para descifrar un archivo previamente cifrado.
- **Funcionamiento:**
 1. Obtiene los parametros pasados por el comando decodificando de base64.
 2. Reconstruye el hash del password usando salt y pbkdf2 para poder reconstruir la clave AES.
 3. Reconstruye la clave AES.
 4. Descripta la clave privada usando la clave AES.
 5. Reconstruye la clave privada con sus bytes.
 6. Reconstruye la clave publica con sus bytes.
 7. Descripta la clave AES del fichero utilizando la clave privada.
 8. Descripta el fichero utilizando la clave AES descriptada del fichero.
 9. Salida JSON (en base64 para guardarlo en la base de datos):

```
{
  "decryptedFile": "<Base64 de bytes descriptados>"
}
```

Modo Share:

- **Comando:** `'java prac1 share <encryptedAESKeyBase64> <password> <saltBase64> <encryptedPrivKeyBase64> <recipientPubKeyBase64>'`
- **Funcionalidad:** Sirve para compartir un archivo cifrado con otro usuario.
- **Funcionamiento:**
 1. Deriva una clave simétrica desde password + salt usando PBKDF2 (200k iteraciones, 256 bits).
 2. Descifra la clave privada RSA del dueño con AES-GCM usando esa clave derivada.
 3. Usa la privada del dueño para descriptar (RSA-OAEP SHA-256) la envoltura de la clave AES del archivo → obtiene la clave AES en claro.
 4. Cifra esa misma clave AES con la clave pública RSA del destinatario (RSA-OAEP SHA-256).
 5. Devuelve JSON con aesKeyReEncrypted (la nueva envoltura de la clave AES, ahora para el destinatario).

Modo Receive

- **Comando:** *java prac1 receive <aesKeyEncrypted> <User2PrivKey> <signature> <timestamp> <user2Pass> <user2Salt> <encryptedFilePath> <user1Pub>'*
- **Funcionalidad:** Sirve para que el destinatario obtenga el archivo compartido, verificando la firma.
- **Funcionamiento:**
 1. Obtiene los datos pasados por parámetro y los decodifica de base64.
 2. Reconstruye el hash del usuario en base al salt y a la contraseña del usuario.
 3. Luego con dicho hash descripta la clave privada del usuario.
 4. Utiliza la clave privada para descriptar la clave AES que ha obtenido el usuario.
 5. Antes de descriptar el archivo verifica que la firma digital es correcta.
 6. Comprueba que la firma digital creada con el archivo encriptado y el timestamp correspondiente es correcto.
 7. Si la firma no es correcta, el método devuelve que la firma es incorrecta y acaba el flujo.
 8. Si la firma es correcta, obtiene el archivo de la carpeta Servidor con la ruta pasada por parámetro.
 9. Una vez obtenido el archivo, lo descripta con la clave AES correspondiente.
 10. Devuelve un JSON con la verificación de la firma y el archivo descriptado.

Funcionamiento de la base de datos

En este apartado vamos a comentar cómo funciona nuestra tabla de datos, debido a que es un elemento muy importante de nuestra aplicación. Nuestra tabla de datos cuenta con tres tablas principales:

1. **Usuario_credentials:** tabla que almacena la información necesaria para autenticar a cada usuario y reconstruir su material criptográfico. Incluye:
 - **Username:** nombre del usuario.
 - **Password_hash + salt:** hash PBKDF2 derivado de su contraseña.
 - **Encrypted_private_key:** clave privada del usuario cifrada (AES).
 - **Public_key:** clave pública RSA del usuario.
 - **Two_factor_secret:** clave para 2FA (si se usa).

Esta tabla es la base del sistema: desde ella se reconstruyen claves y permisos.

2. **User_files:** tabla que registra los archivos cifrados que sube cada usuario junto con su información criptográfica asociada. Incluye:

- **File_name:** nombre del archivo.
- **Encrypted_file_path:** ruta del archivo cifrado almacenado en el servidor.
- **Aes_key_encrypted:** clave AES del archivo cifrada con la clave pública del usuario propietario.
- **FirmaDigitalB64 y firmaTimestamp:** firma digital del archivo original utilizada para verificar su autenticidad e integridad.

Esta tabla representa la relación entre un usuario y todos los archivos cifrados que posee en el sistema.

3. **Shared_files:** tabla que gestiona el intercambio seguro de archivos entre usuarios sin necesidad de recifrar su contenido. Incluye:

- **File_id:** identificador del archivo original almacenado en user_files.
- **Original_owner_id:** usuario que subió el archivo inicialmente.
- **New_owner_id:** usuario que recibe acceso al archivo compartido.
- **Aes_key_encrypted_new_owner:** misma clave AES del archivo, pero cifrada con la clave pública del destinatario.
- **File_path:** ruta del archivo cifrado (compartida con la del original).
- **FirmaDigitalB64 y firmaTimestamp:** firma que permite al receptor verificar la integridad del archivo.
- **Download:** indicador que señala si el nuevo propietario ya ha descargado el archivo.

Esta tabla permite compartir archivos de forma eficiente y segura, evitando duplicaciones y manteniendo la integridad criptográfica.

Conclusión

La implementación de este servicio seguro de contenidos multimedia representó un desafío técnico significativo y una valiosa oportunidad para aplicar principios de criptografía moderna en un entorno real. La fase inicial del proyecto sentó las bases al optar por XAMPP para el entorno de ejecución y el uso de PHP en el backend, un lenguaje que, aunque nuevo para el equipo, permitió conectar la interfaz web con la potente lógica criptográfica desarrollada en Java mediante scripts y la ejecución controlada de comandos shell.

Sin embargo, la segunda parte del proyecto supuso un mayor reto debido a la complejidad técnica y de diseño. Al inicio, nos enfrentamos a la dificultad de distribuir correctamente la aplicación y definir la estructura para escalar las funcionalidades. La implementación de la acción de compartir archivos demostró ser una de las tareas más arduas, pues tuvimos que resolver dudas que nos surgían sobre cómo y dónde

guardar los archivos compartidos sin mezclarlos con los privados, y cómo gestionar diferentes acciones (por ejemplo, qué ocurriría si el archivo original se eliminaba, pero seguía compartido).

A la vez, tuvimos que superar retos importantes en la integración de funcionalidades de seguridad. La implementación del Doble Factor de Autenticación (2FA) requirió entender la conexión con Google Authenticator y la correcta manipulación de las librerías externas (para el protocolo TOTP y la generación del QR). Finalmente, fue un reto de alto nivel criptográfico integrar y validar la Firma Digital para garantizar la integridad y autenticidad de los archivos.

A pesar de estos obstáculos, estamos profundamente satisfechos con el resultado final de la aplicación. Ya que hemos logrado ir más allá de los requisitos iniciales, incorporando seguridad avanzada como SALT y HASH con PBKDF2 para derivar claves seguras a partir de las contraseñas, y hemos resuelto los desafíos planteados por la arquitectura, el 2FA y la firma digital. Este proyecto ha sido una experiencia de aprendizaje invaluable en temas de seguridad, criptografía y arquitectura, y estamos seguros de que el conocimiento práctico adquirido será fundamental al trabajar en futuros proyectos reales. El esfuerzo que hemos invertido a generado sus frutos dando como resultado una aplicación funcional, y alineada con los principios de confidencialidad y seguridad de la información.